

DOCUMENTACIÓN COMPLETA - Sistema IoT de Alarma

ÍNDICE

1. [Resumen del Sistema](#)
 2. [Arquitectura](#)
 3. [Instalación y Configuración](#)
 4. [API Endpoints](#)
 5. [Registro de Cambios](#)
 6. [Despliegue Rápido](#)
 7. [Solución de Problemas](#)
-

1 RESUMEN DEL SISTEMA

Descripción General

Sistema IoT de alarma con doble sensor (PIR + MPU6050) que envía eventos a un backend Node.js, los almacena en PostgreSQL y los visualiza en un dashboard web.

Componentes:

- **ESP32:** Microcontrolador con WiFi
 - Sensor PIR HC-SR501 (movimiento)
 - Sensor MPU6050 (vibración)
 - LED rojo (alarma movimiento)
 - LED amarillo (alarma vibración)
 - Buzzer (alerta sonora)
- **Backend:** Node.js + Express
 - API REST
 - Autenticación con bcrypt
 - Conexión a PostgreSQL
 - Puerto: 3000
- **Base de Datos:** PostgreSQL 15-alpine
 - Docker container
 - Tablas: users, devices, device_events, login_attempts
 - Puerto: 5432
- **Frontend:** HTML + CSS + JavaScript
 - Dashboard en tiempo real

- Visualización de eventos
- Sin framework (vanilla JS)

☑

FUNCIONALIDADES IMPLEMENTADAS

1. Auto-Hash de Contraseñas

- Endpoint: `POST /api/auth/register`
- Las contraseñas se hashean automáticamente con bcrypt

2. API para Eventos del ESP32

- Endpoint: `POST /api/devices/motion`
- Recibe eventos de sensores y los guarda en PostgreSQL

3. Consulta de Historial

- Endpoint: `GET /api/devices/:id/events`
- Obtiene historial de eventos de un dispositivo

4. Usuarios Predeterminados

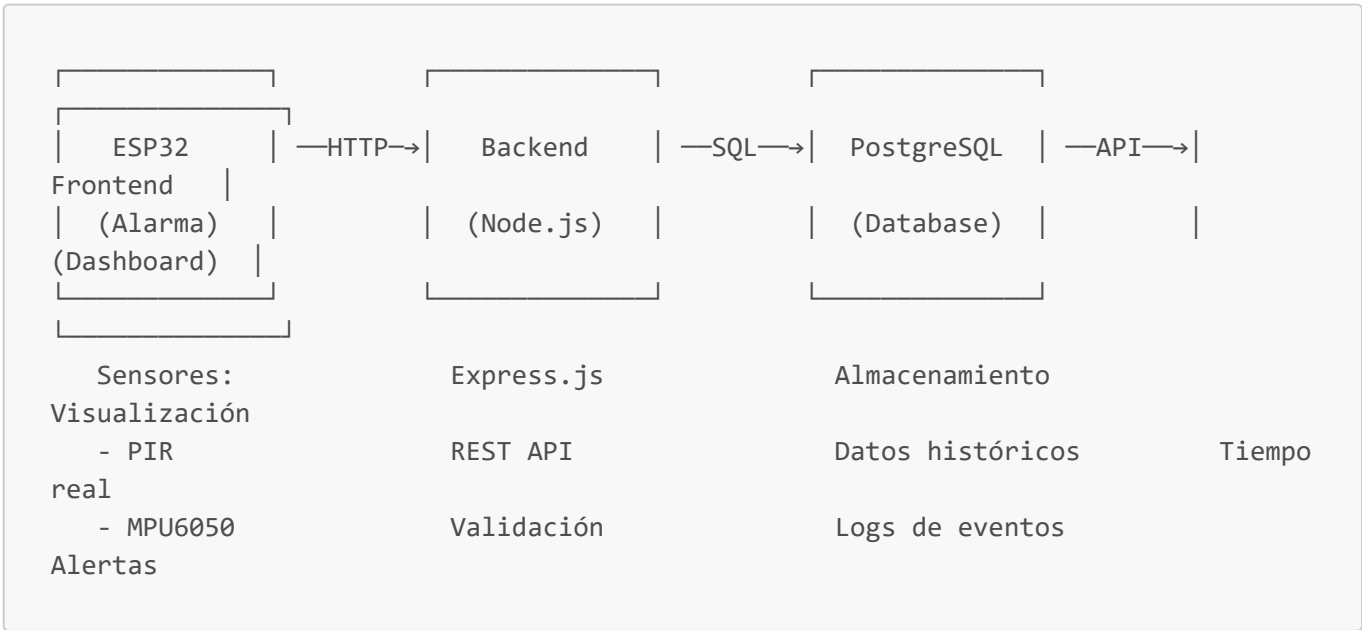
Usuario	Contraseña	Rol
admin	admin123	admin
user	user123	user
demo	demo123	demo
operator	operator123	operator

2

ARQUITECTURA DEL SISTEMA

📊

Flujo de Datos



→SQL→

PostgreSQL

(Database)

Almacenamiento

Datos históricos

Logs de eventos

→API→

Tiempo

☑ ¿POR QUÉ ESTA ARQUITECTURA?

☑ Separación de Responsabilidades:

- **ESP32:** Solo se encarga de sensores y envío de datos
- **Backend:** Validación, lógica de negocio, seguridad
- **PostgreSQL:** Almacenamiento confiable y consultas complejas
- **Frontend:** Interfaz de usuario

☑ Escalabilidad:

- Puedes tener **múltiples ESP32** enviando datos al mismo backend
- PostgreSQL puede manejar **millones de eventos**
- Frontend puede mostrar datos de **todos los dispositivos**

☑ Persistencia:

- Los datos NO se pierden si el ESP32 se reinicia
- Puedes consultar **historial completo** de eventos
- Análisis de patrones y estadísticas

☑ Flexibilidad:

- Cambiar el ESP32 sin afectar el backend
- Actualizar la base de datos sin tocar el ESP32
- Mejorar el frontend independientemente

📄 Flujo Detallado de Eventos

1. ESP32 detecta evento:

```
// Sensor de movimiento (PIR) detecta movimiento
if (sensorStatePIR == HIGH) {
    // Confirma movimiento (750ms)
    // Envía HTTP POST al backend
    sendMotionEvent();
}

// Sensor de vibración (MPU6050) detecta vibración
if (totalAccel > VIBRATION_THRESHOLD) {
    // Detecta vibración
    // Envía HTTP POST al backend
    sendVibrationEvent();
}
```

2. Backend recibe y guarda:

```

POST /api/devices/motion
{
  "device_id": "ESP32_ALARM_01",
  "event_type": "motion_detected",
  "sensor_type": "PIR",
  "timestamp": "2025-10-18 15:30:45",
  "sensor_value": 1
}

// Backend guarda en PostgreSQL
INSERT INTO device_events (device_id, event_type, timestamp, sensor_value)
VALUES ('ESP32_ALARM_01', 'motion_detected', '2025-10-18 15:30:45', 1)

```

3. Frontend consulta datos:

```

// Consulta periódica cada 5 segundos
GET /api/devices/ESP32_ALARM_01/events?limit=10

// Muestra en el dashboard:
// - Últimos eventos
// - Gráficas de actividad
// - Alertas en tiempo real

```

✗ LO QUE NO ES POSIBLE:

ESP32 —SQL directo—→ PostgreSQL ✗

¿Por qué?

- ESP32 no tiene drivers nativos de PostgreSQL
- Consumiría mucha memoria RAM
- Dependencias muy pesadas
- Sin librerías compatibles con Arduino

ESTRUCTURA DE DATOS

Eventos que envía el ESP32:

Movimiento Detectado (PIR):

```

{
  "device_id": "ESP32_ALARM_01",
  "event_type": "motion_detected",
  "sensor_type": "PIR",
  "timestamp": "2025-10-18 15:30:45",

```

```
"sensor_value": 1
}
```

Vibración Detectada (MPU6050):

```
{
  "device_id": "ESP32_ALARM_01",
  "event_type": "vibration_detected",
  "sensor_type": "MPU6050",
  "timestamp": "2025-10-18 15:30:46",
  "sensor_value": 15,
  "additional_data": {
    "acceleration": 15.3,
    "threshold": 12.0
  }
}
```

Alarma Activada:

```
{
  "device_id": "ESP32_ALARM_01",
  "event_type": "alarm_activated",
  "sensor_type": "SYSTEM",
  "timestamp": "2025-10-18 15:30:46",
  "sensor_value": 1,
  "additional_data": {
    "triggered_by": ["PIR", "MPU6050"]
  }
}
```

3 INSTALACIÓN Y CONFIGURACIÓN

Requisitos Previos

- **Node.js** v16 o superior
- **Docker Desktop** (para PostgreSQL)
- **Git** (para clonar repositorio)
- **Arduino IDE** (solo para el compañero con ESP32)

Instalación Paso a Paso

1. Instalar Dependencias del Backend

```
# Navegar a la carpeta del backend
cd "c:\Users\BITGAME\OneDrive\Documentos\GitHub\IoT\Back"
```

```
# Instalar dependencias de Node.js
npm install
```

Dependencias instaladas:

- express v4.21.2
- pg v8.16.3
- bcrypt v5.1.1
- cors v2.8.5
- dotenv v16.4.7

2. Configurar Base de Datos

```
# En la carpeta raíz del proyecto
cd "c:\Users\BITGAME\OneDrive\Documentos\GitHub\IoT"

# Iniciar PostgreSQL con Docker
docker-compose up -d

# Verificar que esté corriendo
docker ps
```

Deberías ver:

CONTAINER ID	IMAGE	PORTS	NAMES
abc123...	postgres:15-alpine	0.0.0.0:5432->5432/tcp	iot_postgres

3. Ejecutar el Backend

```
# Navegar al backend
cd Back

# Iniciar servidor
node server.js
```

Deberías ver:

```
🚀 =====
Backend IoT Server iniciado exitosamente
🚀 =====
📡 Servidor corriendo en: http://localhost:3000
🔗 API disponible en: http://localhost:3000/api
📊 Dashboard: http://localhost:3000/api/dashboard
```

🔑 Auth: <http://localhost:3000/api/auth>
 📱 Devices: <http://localhost:3000/api/devices>

4. Abrir Frontend

```
# Navegar al frontend
cd ../Front

# Abrir en navegador
start index.html
```

📁 Estructura del Proyecto

```
IoT/
├── Back/                                # Backend Node.js
│   ├── package.json                    # Dependencias
│   ├── server.js                       # Servidor principal
│   ├── database.js                     # Conexión PostgreSQL
│   ├── hashPassword.js                 # Utilidad para hashear
│   └── routes/
│       ├── auth.js                     # Autenticación
│       ├── dashboard.js                 # Dashboard API
│       └── devices.js                   # CRUD dispositivos
├── BaseDeDatos/                         # PostgreSQL
│   ├── package.json
│   ├── postgresql.js
│   └── init/
│       └── 02-consultas.sql             # Esquema de BD
├── Front/                               # Frontend
│   ├── index.html                      # Dashboard principal
│   ├── login.html                      # Página de login
│   ├── script.js                       # Lógica del dashboard
│   ├── login.js                         # Lógica de login
│   └── styles.css                       # Estilos
├── Esp32/                               # Código ESP32
│   ├── INSTRUCCIONES_PARA_COMPañERO.md
│   └── V2_ambosSensores/
│       ├── V3_ConBackend.ino           # Código principal
│       └── config.h                     # Configuración WiFi
├── docker-compose.yml                   # Docker PostgreSQL
└── .gitignore                           # Archivos ignorados
```

4 API ENDPOINTS

Autenticación

POST /api/auth/login

Login de usuario.

Request:

```
{
  "username": "admin",
  "password": "admin123"
}
```

Response:

```
{
  "success": true,
  "message": "Login exitoso",
  "user": {
    "id": 1,
    "username": "admin",
    "email": "admin@iot.com",
    "role": "admin"
  }
}
```

POST /api/auth/register

Registrar nuevo usuario (auto-hash de contraseña).

Request:

```
{
  "username": "nuevo_usuario",
  "password": "miContraseña123",
  "email": "usuario@example.com",
  "role": "user"
}
```

Response:

```
{
  "success": true,
  "message": "Usuario registrado exitosamente",
}
```



```
"user": {
  "id": 5,
  "username": "nuevo_usuario",
  "email": "usuario@example.com",
  "role": "user"
}
```

Dashboard

GET /api/health

Estado del servidor y conexión a BD.

Response:

```
{
  "status": "ok",
  "timestamp": "2025-10-18T15:30:45.123Z",
  "database": "connected",
  "uptime": 3600
}
```

GET /api/dashboard

Datos completos del dashboard.

Response:

```
{
  "success": true,
  "data": {
    "kpis": { /* ... */ },
    "temperatureHistory": [ /* ... */ ],
    "devices": [ /* ... */ ]
  }
}
```

Dispositivos (Devices)

GET /api/devices

Listar todos los dispositivos.

Response:

```
{
  "success": true,
  "count": 5,
  "data": [
    {
      "id": "ESP32_ALARM_01",
      "name": "Sensor Principal",
      "type": "ESP32",
      "status": "online",
      "last_seen": "2025-10-18T15:30:45.123Z"
    }
  ]
}
```

POST /api/devices/motion

Recibir evento de movimiento/vibración desde ESP32.

Request:

```
{
  "device_id": "ESP32_ALARM_01",
  "event_type": "motion_detected",
  "sensor_type": "PIR",
  "timestamp": "2025-10-18 15:30:45",
  "sensor_value": 1,
  "additional_data": {
    "confirmed": true
  }
}
```

Response:

```
{
  "success": true,
  "message": "Evento guardado exitosamente",
  "event": {
    "id": 123,
    "device_id": "ESP32_ALARM_01",
    "event_type": "motion_detected",
    "timestamp": "2025-10-18T15:30:45.000Z"
  }
}
```

GET /api/devices/:deviceId/events

Obtener historial de eventos de un dispositivo.

Query Parameters:

- **limit** (default: 50) - Número máximo de eventos
- **offset** (default: 0) - Desplazamiento para paginación

Ejemplo:

```
GET /api/devices/ESP32_ALARM_01/events?limit=10&offset=0
```

Response:

```
{
  "success": true,
  "count": 10,
  "data": [
    {
      "id": 123,
      "device_id": "ESP32_ALARM_01",
      "event_type": "motion_detected",
      "sensor_type": "PIR",
      "timestamp": "2025-10-18T15:30:45.000Z",
      "sensor_value": 1
    }
  ]
}
```

5 REGISTRO DE CAMBIOS

 Versión 3.0 - Integración ESP32 (Octubre 2025)

☒ Nuevas Funcionalidades:

1. Código ESP32 con Backend

- **V3_ConBackend.ino** - Código completo con WiFi
- Envío de eventos HTTP al backend
- Manejo de reconexión WiFi automática
- Sincronización de tiempo NTP

2. API de Eventos

- POST **/api/devices/motion** - Recibir eventos
- GET **/api/devices/:id/events** - Historial de eventos
- Soporte para datos adicionales (JSON)

3. Base de Datos Mejorada

- Nueva tabla `device_events`
- Campo `additional_data` (JSONB)
- Índices optimizados

Versión 2.0 - Seguridad (Octubre 2025)

Mejoras de Seguridad:

1. Bcrypt para Contraseñas

- Instalado `bcrypt v5.1.1`
- Auto-hash en registro de usuarios
- Verificación segura en login

2. Utilidad de Hashing

- Script `hashPassword.js`
- Generar y verificar hashes

3. Usuarios Predeterminados

- 4 usuarios con contraseñas hasheadas
- Roles: admin, user, demo, operator

Versión 1.0 - Base (Septiembre 2025)

Funcionalidades Iniciales:

1. Backend Node.js + Express

- API REST completa
- CRUD de dispositivos
- Dashboard endpoints

2. PostgreSQL en Docker

- Tablas: users, devices, login_attempts
- Datos de ejemplo

3. Frontend HTML/CSS/JS

- Dashboard responsive
- Login system
- Visualización en tiempo real

GUÍA DE DESPLIEGUE RÁPIDO

COMANDOS PARA EJECUTAR TODO

Terminal 1: Base de Datos

```
cd C:\Users\BITGAME\OneDrive\Documentos\GitHub\IoT
docker-compose up -d
```

☒ Puedes cerrar esta terminal después

Terminal 2: Backend ⚠ MANTENER ABIERTA

```
cd C:\Users\BITGAME\OneDrive\Documentos\GitHub\IoT\Back
node server.js
```

⚠ NO CERRAR - Debe quedarse corriendo

Terminal 3: Frontend

```
cd C:\Users\BITGAME\OneDrive\Documentos\GitHub\IoT\Front
start index.html
```

☒ Puedes cerrar esta terminal después

🔄 ORDEN CORRECTO DE EJECUCIÓN

1	docker-compose up -d	(PostgreSQL)
	↓	
2	node server.js	(Backend) ⚠ DEJAR ABIERTO
	↓	
3	start index.html	(Frontend)
	↓	
4	Arduino IDE → Upload	(ESP32) - Lo hace tu compañero

☒ VERIFICACIÓN RÁPIDA

```
# Verifica que todo esté corriendo:

# PostgreSQL (debería mostrar el contenedor)
docker ps

# Backend (debería responder)
curl http://localhost:3000/api/health

# Frontend (debería abrir en navegador)
start http://localhost:3000
```

CHECKLIST DE VERIFICACIÓN

- ☐ Docker Desktop corriendo
- ☐ PostgreSQL container activo (`docker ps`)
- ☐ Backend corriendo sin errores
- ☐ Frontend accesible en navegador
- ☐ Login funciona con usuarios predeterminados
- ☐ API `/api/health` responde correctamente

7 SOLUCIÓN DE PROBLEMAS

✗ PROBLEMA: PostgreSQL no conecta

Error:

```
la autenticación password falló para el usuario iot_user
```

Solución 1: Recrear Docker

```
cd C:\Users\BITGAME\OneDrive\Documentos\GitHub\IoT
docker-compose down -v
docker-compose up -d
```

Solución 2: Usar trust authentication (solo desarrollo)

```
# En docker-compose.yml, descomentar:
POSTGRES_HOST_AUTH_METHOD: trust
```

Solución 3: Seguir con datos simulados

- El backend tiene fallback a datos simulados
- Funciona perfectamente sin PostgreSQL
- Arreglar después cuando sea necesario

✗ PROBLEMA: Backend no arranca

Error:

```
Cannot find module 'bcrypt'
```

Solución:

```
cd Back
npm install bcrypt
npm install
```

✗ PROBLEMA: ESP32 no envía datos

Posibles causas:

1. WiFi no conectado

- Verificar contraseña en `config.h`
- Verificar SSID correcto ("Altro52.4G")
- Acercarse al router

2. Backend no corriendo

- Verificar que `node server.js` esté activo
- Verificar IP correcta (192.168.1.138)

3. Librería faltante

- Instalar ArduinoJson en Arduino IDE
- Sketch → Include Library → Manage Libraries
- Buscar "ArduinoJson" → Instalar

✗ PROBLEMA: Frontend no carga datos

Solución:

```
// Verificar en consola del navegador (F12)
// Debería ver:
console.log('API Response:', data);

// Si hay error CORS:
// Verificar que backend tenga CORS habilitado
```

✗ PROBLEMA: Puerto 3000 ya en uso

Solución:

```
# Matar proceso en puerto 3000
netstat -ano | findstr :3000
taskkill /PID <PID_NUMBER> /F

# O cambiar puerto en server.js
const PORT = 3001; // Cambiar puerto
```

SOPORTE Y RECURSOS

Documentación de Referencia

- **Node.js**: <https://nodejs.org/docs>
- **Express.js**: <https://expressjs.com>
- **PostgreSQL**: <https://www.postgresql.org/docs>
- **ESP32**: <https://docs.espressif.com>
- **Arduino IDE**: <https://www.arduino.cc/reference>

Herramientas Útiles

- **Postman**: Para probar API endpoints
- **DBeaver**: Para gestionar PostgreSQL
- **Serial Monitor**: Para debug del ESP32

Archivos Importantes

- [INSTRUCCIONES_PARA_COMPañERO.md](#) - Guía para el compañero con ESP32
- [.gitignore](#) - Archivos a ignorar en git
- [docker-compose.yml](#) - Configuración de PostgreSQL
- [package.json](#) - Dependencias del proyecto

¡PROYECTO COMPLETO!

Tu sistema IoT está listo con:

- ☒ Backend seguro con bcrypt
- ☒ API REST completa
- ☒ Base de datos PostgreSQL
- ☒ Frontend dashboard
- ☒ Código ESP32 con integración
- ☒ Documentación completa

¡Éxito con tu proyecto! 

Documento generado el 18 de octubre de 2025 Versión 3.0 - Documentación Completa